DESIGN AND DEVELOPMENT OF NEXUS-AI

An AI based UNIVERSAL PROBLEM SOLVING SYSTEM(UPSS)

Team-DATAFORGE

Corner Stone Project Report

Milestone III

Submitted By

Vansh Pratap Singh Jadon (BTCS24O1145)

Faculty Mentor

Dr. Kirti Raj Bhatele

Assistant Professor

Department of Computer Science & Engineering



माधव प्रौद्योगिकी एवं विज्ञान संस्थान, ग्वालियर (म.प्र.), भारत

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR (M.P.), INDIA

Deemed University

(Declared under Distinct Category by Ministry of Education, Government of India)

NAAC ACCREDITED WITH A++ GRADE

July-Dec 2026

1. Objective

The objective is to transform the **structured execution plan** generated by the Planner into an executable workflow.

The system dynamically:

- Generates suitable agents for planned tasks
- Identifies required capabilities
- Selects appropriate tools
- Selects available providers
- Executes tools through a standardized interface
- Supports sequential, parallel, and dependency-based execution

This creates the bridge between **planning and actual task execution**.

2. Agent Team Generation

After planning, each PlannerTask is analyzed to determine the type of agent required.

Execution Plan

↓

Task Analysis

↓

Required Capabilities

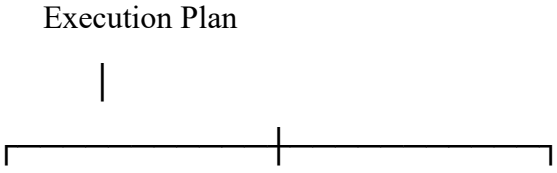
↓

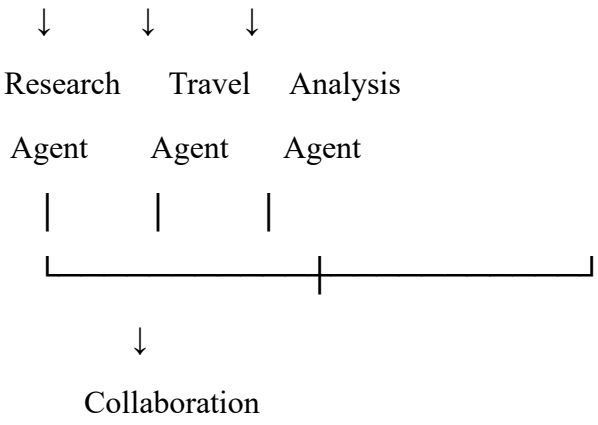
Agent Selection

↓

Agent Team

For a complex request, multiple specialized agents can be created:





Agents can execute independently or exchange intermediate results depending on task dependencies.

3. Dynamic Tool Selection

The Tool Selection layer converts task requirements into actual executable tools.

PlannerTask

↓

CapabilityMatcher

↓

ToolMatcher

↓

ToolRanker

↓

ProviderSelector

↓

Selected Tool + Provider

The ToolSelectorAgent coordinates these components:

```python
capabilities = self.capability_matcher.match(task)

candidates = self.tool_matcher.match(
    capabilities
)

tool = self.tool_ranker.rank(
    candidates
)

provider = self.provider_selector.select(
    tool
)
```

The final selection contains:

```python
return {
    "tool": tool,
    "provider": provider,
    "capabilities": capabilities,
}
```

This keeps the planner independent from specific tools and providers.

4. Dynamic Tool Discovery & Registry

Tools are automatically discovered from the tools package instead of maintaining a hardcoded tool list.

```python
for module in pkgutil.walk_packages(
    [package_path],
    prefix=f"{package}.",
):
    imported_module = importlib.import_module(
        module.name
    )
```

Every class derived from BaseTool can be discovered and registered:

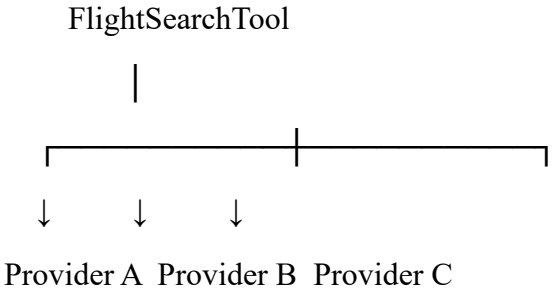
```python
if (
    issubclass(obj, BaseTool)
    and obj is not BaseTool
):
    tool = obj()

    self.registry.register_tool(
        name=tool.name,
        tool=tool,
        domain=tool.domain,
        capabilities=tool.capabilities,
        providers=tool.providers,
        enabled=tool.enabled,
    )
```

This makes the tool ecosystem **extensible and dynamically discoverable**.

5. Provider Selection & Execution

A tool can support multiple providers. The ProviderRegistry maintains these relationships.



The provider selector chooses an available provider for the selected tool.

The execution path is:

Selected Tool

↓

ToolManager

↓

ToolExecutor

↓

ProviderFactory

↓

Provider

↓

External API / Service

↓

ToolResult

The ToolManager provides the high-level execution interface:

```python
async def execute(
    self,
    tool_name: str,
    context: ToolContext,
    request: ToolInput,
) -> ToolResult:

    return await self.executor.execute(
        tool_name=tool_name,
        context=context,
        request=request,
    )
```

6. Standardized Tool Lifecycle

All tools follow a common execution model through BaseTool.

Tool Request

↓

Initialize

↓

Validate

↓

Before Execute

↓

Execute

↓

After Execute

↓

Tool Result

Example:

```python
if not self._initialized:
    await self.initialize()

await self.validate(request)

await self.before_execute(context)

result = await self.execute(
    context=context,
    request=request,
)

await self.after_execute(
    context=context,
    result=result,
)
```

This provides a consistent execution contract for different types of tools.

7. Execution Strategies

The execution engine determines how tasks should run based on their dependencies.

Sequential Execution

Task A

↓

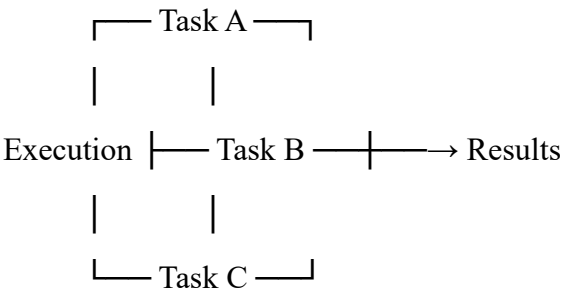
Task B

↓

Task C

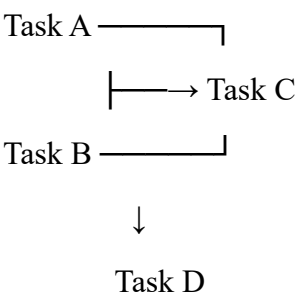
Used when each task depends on the previous result.

Parallel Execution



Independent tasks can execute simultaneously.

Dependency-Based Execution



Tasks execute only when their required dependencies are available.

8. Complete Execution Workflow

USER REQUEST

↓

PLANNER

↓

PlannerTask

↓

Agent Generation

↓

Capability Matching

↓

Tool Matching

↓

Tool Ranking

↓

Provider Selection

↓

Tool Manager

↓

Tool Executor

↓

Provider Factory

↓

Real Provider/API

↓

ToolResult

↓

Agent Collaboration

↓

Execution Results

9. Key Outcome

This implementation establishes the complete **Agent → Tool → Provider → Execution** pipeline.

The planned tasks can now be mapped to suitable execution resources dynamically rather than relying on fixed tool assignments.

Core achievement:

PlannerTask → Agent → Capability → Tool → Provider → Execution → ToolResult